Replication materials are great but software versioning still poses a problem for open science

Taylor J. Wright* Bruno Rodrigues †

November 9, 2023

*Brock University, twright3@brocku.ca
†Ministry of Higher Education and Research, Luxembourg

0.1 Introduction

Beginning in the early 2010s with psychology and quickly spilling over into political science and economics on the backs of high-profile cases of fraud (Broockman, Kalla, and Aronow (2015)) and errors (Herndon, Ash, and Pollin (2014)), the "replication crisis" in the social sciences has become mainstream, even finding its way into the popular press (Aschwanden (2015), Gelman (2018)). Concerns over how well published results hold up to replication in other settings has led to large scale initiatives to document how trustworthy the existing stock of evidence is (e.g. Camerer et al. (2016), Chang and Li (2022), and Collaboration (2015)). While the *replicability* of studies has received much attention, there has been a tandem movement discussing the importance of and need for research *reproducibility*. While the exact scope and definition is subject to ongoing debate, replication is, broadly speaking, re-testing a hypothesis while changing an element of previous research (e.g. the sample or estimating equation) whereas reproducibility (sometimes referred to as verification or computational reproducibility) is following a study's protocol or methods exactly and obtaining the results presented in the study.[1]

In our view, reproducibility is an insufficient but necessary condition for replication—that is, work cannot be replicable if it is not reproducible and it likely does not make sense to spend resources on replication if research is not in the first place reproducible.

These concerns about the reproducibility (and replicability) of social science research have prompted a push, often referred to as Data Access and Research Transparency (DA-RT) in political science following Lupia and Elman (2014), for journals to require the publication of research materials that accompany academic research.[2] Specifically, the provision and publication of underlying data and code used for data preparation and analysis. This push has seen some success, with a growing number journals now requiring the provision of replication materials as a condition of publication. Some journals, such as the American Economic Review (AER) or American Journal of Political Science (AJPS), even have verification policies that require authors to upload their replication materials and have their results verified by another team (either the journal's replication team (AER) or a third party, (AJPS)) before publication. However, even if these materials are provided, and even when in place

[1] For further discussion on definitions, scopes, and types of replication and reproducibility see, for example, Clemens (2017), Derksen and Morawski (2022), Hamermesh (2007), and Nosek and Errington (2020).

[2] See, for example, the 2014 Joint Editors Transparency Statement which was signed by editors of 27 leading political science journals: https://www.dartstatement.org/2014-journal-editors-statement-jets

these policies do not have perfect compliance (Philip (2010), Stockemer, Koehler, and Lentz (2018)), regular software updates and new version releases can result in the replication materials failing to faithfully reproduce the authors' results or even run at all. [3]

In this paper, we present a case study of an article published in Journal of Politics in January 2022 titled, "Multiracial Identity and Political Preferences", Davenport, Franco, and Iyengar (2022), that details replication challenges arising from changes in the statistical software R. We were unable to reproduce the authors' results using either the current version of R, or the version that the authors indicate they used. The lack of reproducibility arose due to a change in the defaults used by base R when generating random numbers starting in version 3.6.0.

We contribute to the existing literature in three ways. First, we add to the discussion about the importance of the availability of replication materials. Existing research documents that the availability of replication materials is associated with a higher citation count (Stockemer, Koehler, and Lentz (2018), is this reference true?) and with a higher likelihood of replication (Philip (2010)). Much of this research focuses on the provision of data and code as a necessary condition for facilitating replication and rebuilding trust in science (). We contribute to this literature by emphasizing that, while we agree that data and code availability are necessary, their existence cannot guarantee reproducibility. This brings us to our second contribution: we expand on the discussion of problems raised by software and package versioning for reproducibility by providing a walkthrough of the problem using a concrete example. Lastly, we further the discussion of tools and best practices for ensuring reproducibility. Specifically, we provide a step-by-step guide to using Docker, {renv}, and {targets} to ensure the reproducibility of research.

The rest of the article proceeds as follows: Section 2 walks through the reproducibility issues in Davenport, Franco, and Iyengar (2022); Section 3 discusses currently available tools and best practices (e.g. Docker and R packages such as `renv`, `groundhog`) for ensuring that replication materials continue to faithfully reproduce research results, despite post-publication changes in the tools used; and Section 4 concludes.

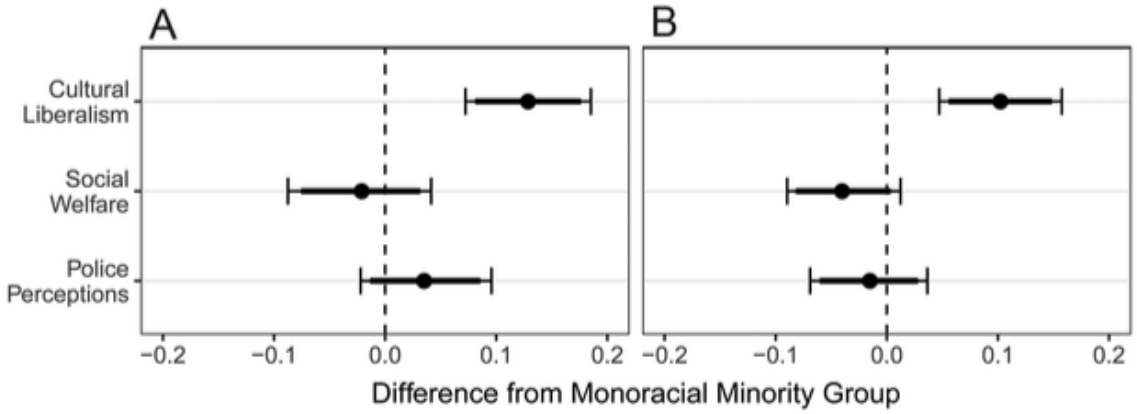
[3] Simonsohn (2021) presents several examples of R changes that could break scripts.

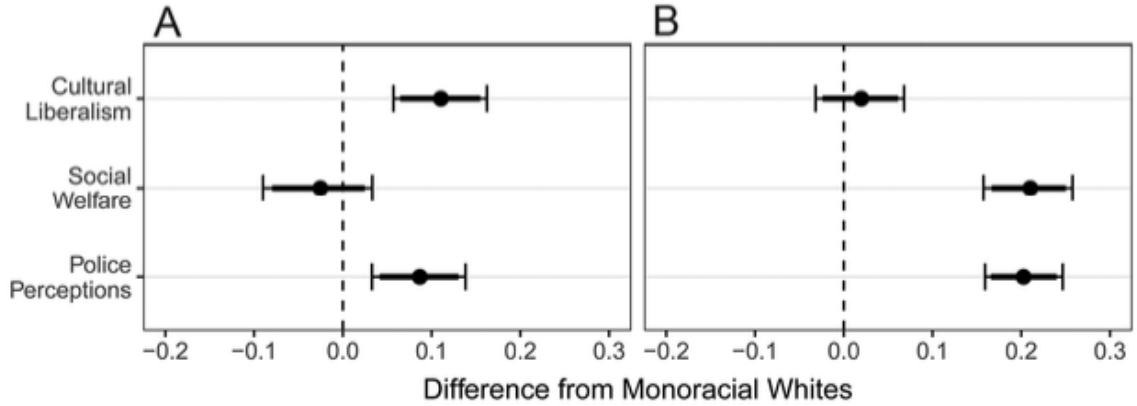
0.2 Reproduction

0.2.1 Illustrating the issue with software versioning

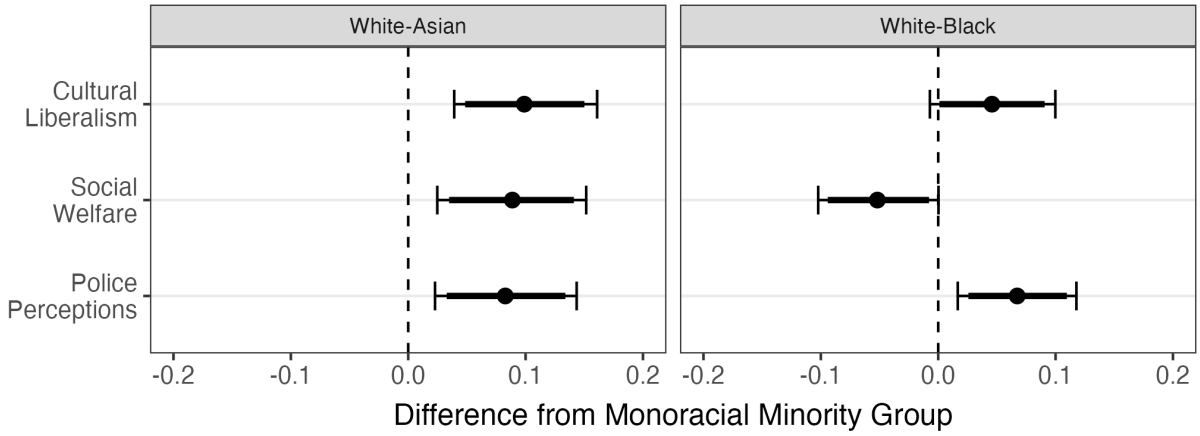
A key thing here is that I don't think we want to be too hostile sounding towards these authors, they had readme files and reproducibility materials available. It's just that manual entry human-error hobgoblins got them with the R versioning.

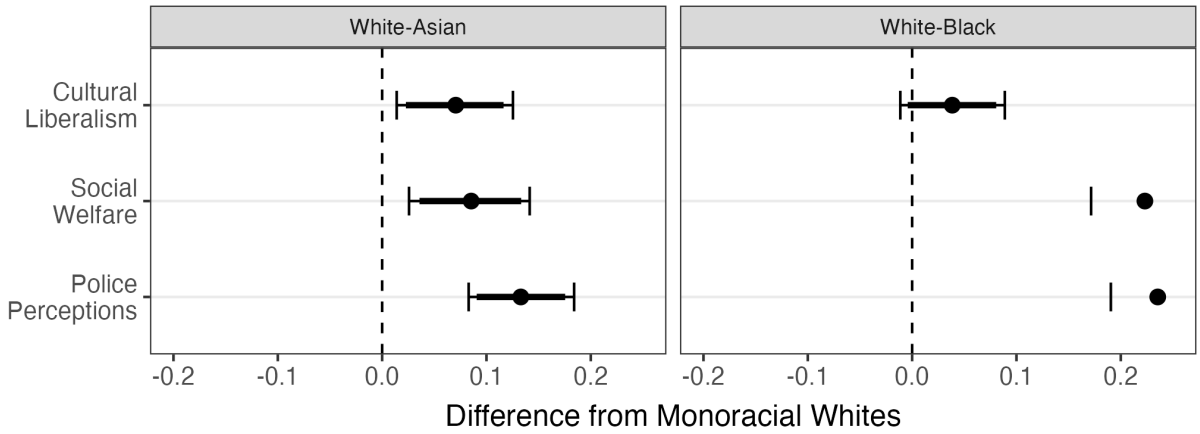
- Brief discussion of authors' paper and context
- The results of their code using stated software version in documentation (just screenshot right now)



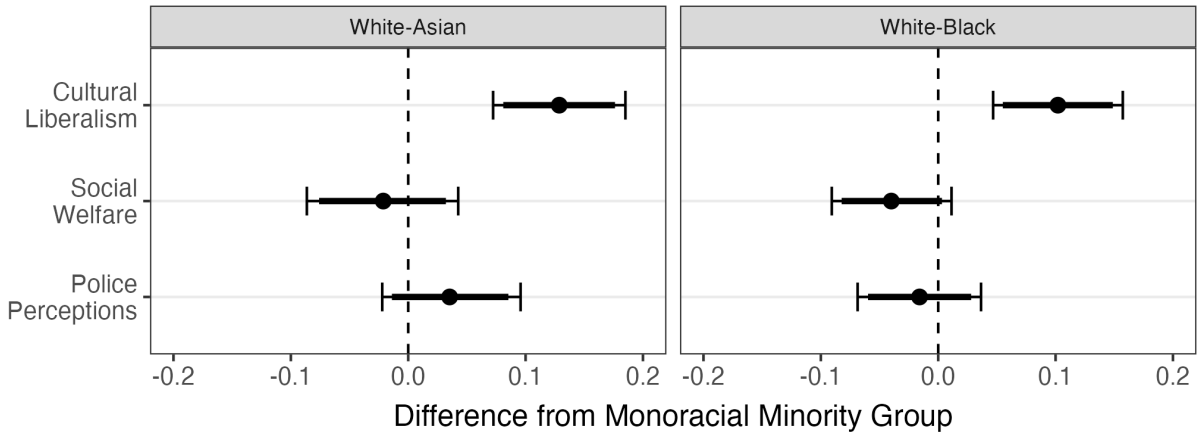


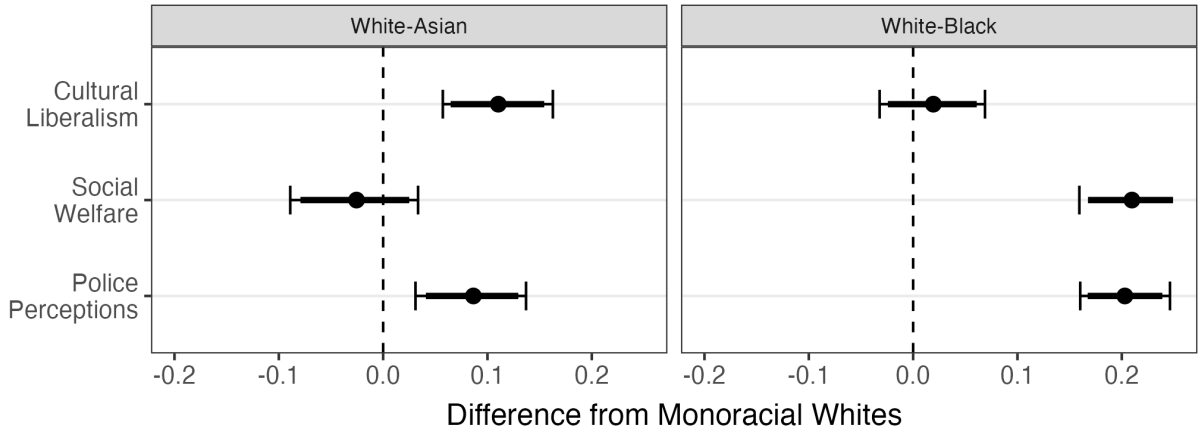
- The results of their code using later software version (post 3.6.0)





- Results of their code using earlier software version (pre 3.6.0)





The authors note that they use R version 3.6.2. Issue seems to be the weights the authors use are non-integer and Zelig uses sample() in that case which following changes to base R yields different results in 3.6.x and 3.5.x (see http://docs.zeligproject.org/articles/weights.html). Prior to R 3.6.x RNGkind(sample.kind = "Rounding") was the default behaviour but after 3.6.0 the sample function's new default behaviour is RNGkind(sample.kind = "Rejection") (see https://blog.revolutionanalytics.com/2019/05/whats-new-in-r-360.html).

Soemthing about how there are ways to ensure that the estimates are consistent (changing the weights to integers or setting the RNGkind to be backwards compatible) but documenting the correct version of R used in the analysis is probably the easiest way. Segue into…

0.3 Discussion

0.3.1 Rebuilding the original development environment using Docker

Replicating results from past studies is quite challenging, for many reasons. This article focuses on one of these reasons: results cannot be reproduced because of changes introduced in more recent versions of the software used for analysis, despite the availability of both data and the authors' original scripts.

To reproduce the results from the original study and to pinpoint the impact of the change introduced in R 3.6.0, we chose to use Docker. Docker is a containerization tool which enables one to build so-called *images*. These images are usually built on top of a Linux operating system and contain all of the required software to build any type of output. After the image containing the output has been built, users in turn only need to be able to run Docker *containers* instantiated from the image definition in order to run the output. Containerization tools such as Docker solved the "works on my machine" problem: this problem arises when software that works well on the development machine of the developer fails to run successfully on a customer's machine. This usually happens because the customer's computer does not have the necessary dependencies to run the software (which are traditionally not shipped alongside the software product) or because of version mismatch of the operating system.

The same approach can be used to build research projects, which also suffer from the same "works on my machine" problem as any other type of software. Containerization tools offer a great opportunity for reproducibility in research: instead of just sharing a replication script, authors can now easily share the right version of the software used to produce these scripts, as well as the right version of the used libraries by building and sharing a Docker image (or at least provide the necessary blueprint to enable others to do so, as we will discuss below). Future researchers looking to reproduce the results can now simple run a container from the provided image (or build an image themselves if the original authors provided the required blueprints).

Concretely, to build a Docker image, a researcher writes a so-called *Dockerfile*, which lists all the necessary steps to build an output. Here is an example of a very simple Dockerfile:

```
FROM rocker/r-ver:4.3.0
```

```
CMD ["R"]
```

This Dockerfile contains two lines: the first line states which Docker image we are going to use as a base. Our image will be based on the `rocker/r-ver:4.3.0` image. The Rocker project is a repository containing many images that ship different versions of R and packages pre-installed: so the image called `r-ver:4.3.0` is an image that ships R version 4.3.0. The last line states which command should run when the user runs a container defined from our image, so in this case, simply the R interactive prompt. Concretely, the above image would allow someone to run R version 4.3.0 as a container, completely isolated from the rest of their computer. This allows users to run different versions of R on the same system. We can add more commands in order to run a complete research project, from start to finish. Below is an example of a Dockerfile that runs an analysis script, by first stating which base image is going to be used, and then adding packages, files and data to run the entire project:

```
FROM rocker/r-ver:4.3.0

RUN R -e "install.packages('dplyr')"

RUN mkdir /home/research_project

RUN mkdir /home/research_project/data

RUN mkdir /home/research_project/project_output

RUN mkdir /home/research_project/shared_folder

COPY analyse_data.R /home/research_project/analyse_data.R

COPY dataset.csv /home/research_project/dataset.csv
```

```
RUN cd /home/research_project && R -e "source('analyse_data.R')"

CMD mv /home/research_project/project_output/* /home/research_project/shared_folder/
```

This Dockerfile starts off from the same base image, an image that ships R version 4.3.0, then it installs the `{dplyr}` package, a popular R package for data manipulation and it creates three directories:

- `/home/research_project`
- `/home/research_project/project_output`
- `/home/research_project/shared_folder`.

Then, it copies the `analyse_data.R` script , which contains the actual analysis made for the purposes of the research project, and the `dataset.csv` file, into the Docker image. The second-to-last line runs the script, and the last line moves the outputs generated from running the `analyse_data.R` script to a folder called `shared_folder`. It is important to say that `RUN` statements will be executed as the image gets built, and `CMD` statements will be executed as a container runs. Using this Dockerfile, an image called `research_project` can be built with the following command:

```
docker build -t research_project .
```

This image can then be archived and shared for replication purposes. Other images can be built on top of this one, and will thus contain R version 4.3.0, the `{dplyr}` package, the data and the project's folder structure. Future researchers can also run a container from that image using a command such as:

```
docker run -d -it --rm --name research_project_container \
  -v /host/machine/shared_folder:/home/research_project/shared_folder:rw \
  research_project
```

The container, called `research_project_container` will execute the `CMD` statement from the Dockerfile, in other words, move the outputs to the `shared_folder`. This folder is like a tunnel between the machine that

runs the container and the container itself: by doing this, the outputs generated within the container can now be accessed from the host's computer.

The image built by the process above is immutable: so as long as users can run it, the outputs produced will be exactly the same as when the original author ran the original analysis. However, if the image gets lost, and needs to be rebuilt, the above `Dockerfile` will not generate the same image. This is because the `Dockerfile`, as it is written above, will download the version of `{dplyr}` that is current at the time it gets built. So if a user instead builds the image in 5 years, the version of `{dplyr}` that will get downloaded will not be the same as the one that was actually used for the original analysis. The version of R, however, will forever remain at version 4.3.0.

So to ensure that future researchers will download the right versions of packages that were originally used for the project, the original researcher also needs to provide a list of packages that were used as well as the packages' versions. This can be quite tedious if done by hand, but thankfully, there are ways to generate such lists very easily. The `{renv}` package for the R programming language provides such a function. Once the project is done, one simply needs to call:

```
RUN rig install 4.2.0

RUN rig default 4.2.0
```

Everything else: package versions and operating system that the replication script runs on, stayed the same.

With this setup, we were thus able to run the original analysis on an environment that was as close as possible to the original environment used by Davenport, Franco, and Iyengar (2022). However, it is impossible to regenerate the exact environment now. As stated, we had to make an assumption on the date the packages were downloaded, but the packages use by the original authors might have been much older. Another likely difference is that the operating system used inside Docker is the Ubuntu Linux distribution. While Ubuntu is a popular Linux distribution, it is much more likely that the original authors used either Windows or macOS to perform their analysis. In most cases, the operating system does not have an impact on results, but there have been replication studies that failed because of this, as in Bhandari Neupane et al. (2019). Thankfully, mismatch of the operating system does not seem to be an issue here.

It should also be noted that strictly speaking, using Docker is not completely reproducible either: this is because the underlying base image, upon which we build our complete analysis, also changes through time. In the examples above the image we used, `rhub/rig:latest` is based on Ubuntu 22.04, which is, at of writing, the latest long-term support release of the Ubuntu operating system. This version of Ubuntu thus still gets updated: so when building the image for our project, the underlying base image will be different today than from 6 months from now. And when the next Ubuntu long-term support will be released, in April 2024, then `rhub/rig:latest` will this time be based upon this new release of Ubuntu. So building in May 2024 will yield a different image than now. In order to avoid this, it is possible to use a so-called *digest* which ensure that exactly the same image will forever be used as a base. For the purposes of this paper we used the following image: `rhub/rig@sha256:0667321839be0c2ade5c14fd8c8bbb311a843ddf690081c5d0dd80dcc98bd6c6`.

Finally, to ensure completeness, instead of using the Posit Package Manager with a fixed date, we could have used the `{groundhog}` package by Simonsohn (2023). This package makes it possible to download packages as of a certain date as well, and it does not require users to change any of R's default configurations.

[4] https://github.com/r-lib/rig
[5] https://packagemanager.posit.co/client/#/repos/2/overview

0.3.2 Reproducibility isn't just version and package management — using {targets} to improve readibility and reproducibility

Reproducibility should go beyond providing the right versions of the analysis software used. Another import aspect is *readability*, which is difficult to quantify. There are however some approaches that we suggest researchers could employ to improve the readability of their code and thus increase the probability of a successful replication.

Most research papers and studies' computer code is written as one, or several, scripts that must be run in a certain order. These script-based workflows usually grow very large and become chaotic. Documentation must be written alongside the scripts to explain how they work and in which order they're supposed to be executed, and this documentation then has to be maintained and updated as the scripts evolve. When some parts of the code gets changed in one script, the researcher has to remember which parts of the other scripts get impacted and either only run the relevant, now outdated, parts, or re-run the whole project which can be quite time-consuming. Here again, it helps to view a researcher paper (and in particular its computer code) as a piece of software. Software engineers are faced with the exact same problem but solved it many decades ago using so-called build automation tools. These build automation tools allow one to describe how a particular piece of software should get built. Other software engineers that wish to build that piece of software thus only need to execute the build automation tool which will take care of executing the right steps in the right order.

The solution we propose is for researchers to use build automation tools. For the R programming language, a very popular build automation tool is provided through the `{targets}` package by Landau (2021). Using `{targets}` has many benefits:

- `{targets}` keeps track of all the inter-dependencies between the different pieces of the entire codebase. If some part gets changed, `{targets}` *knows* which other parts are affected and only re-executes these;
- another consequence of this is that `{targets}` also knows which parts of the codebase are independent from each other and can thus be safely executed in parallel. This can lead to tremendous execution speed gains;
- `{targets}` works by having the user define the computations as a pipeline. This pipeline is in essence the composition of many pure functions, which increases the readibility of the project.

As an illustration of this, we rewrote parts of the original study as a {targets} pipeline, by re-using the original code from the authors and rewriting only what was needed to "port" it to {targets}. A {targets} pipeline is defined by at least one file called _targets.R. Within this file, a list of targets objects is defined. For example, here is the beginning of our _targets.R that we had to write for our purposes (not shown are the calls to load required libraries and helper functions):

```
list(
  tar_target(
    data_path,
    "/home/r_and_p/data/multiracial.sav"
  ),
  tar_target(
    multiracial_raw,
    read_sav(data_path)
  ),
  tar_target(
    multiracial_1,
    basic_clean_1(multiracial_raw)
  ),
  tar_target(
    multiracial_2,
    closeness_2(multiracial_1)
  ),
```

The first target, data_path simply defines the path to the data set from the original study. The second target, multiracial_raw is the data loaded into memory. This second target is defined as the output of the read_sav(data_path) command. Because this second target uses the first target as an input, {targets} is able to detect that these targets are linked. The third target is multiracial_1, which is defined as the output of a function called basic_clean_1(). This is a function that we defined into another script called functions.R. This script contains all the code from the original study that was written to clean the data.

The only difference is that we have re-defined every piece of code into a function. For example, this is what `basic_clean_1()` looks like:

```
basic_clean_1 <- function(dataset){
  dataset %>%
      clean_names() %>%
      mutate(across(where(is.labelled),
                    as_factor)) %>%
      # Exclude South Asians
      filter((is.na(q24_2) | q24_2 == "No") &
             (is.na(q24_3) | q24_3 == "No")) %>%
      mutate(educ3 = case_when(
               educ %in% c("No HS", "High school graduate") ~
                 "Less than or equal HS",

               educ == "Some college" ~
                 "Some college",

               educ %in% c("2-year", "4-year", "Post-grad")  ~
                 "College Grad",
               TRUE ~ NA_character_
             )) %>%
    mutate(age = 2015 - as.numeric(as.character(birthyr))) %>%
    mutate(income = if_else(faminc == "Prefer not to say", NA, faminc),
           income = fct_lump_min(income, 100,
                                 other_level = ">150,000")) %>%
    mutate(south = ifelse(
             inputstate %in% c("Alabama", "Arkansas", "Florida",
                               "Georgia", "Kentucky", "Louisiana",
                               "Mississippi", "North Carolina",
```

```
                                "South Carolina", "Tennessee", "Texas",
                                "Virginia", "West Virginia"),
              1, 0
            ))
}
```

and here is the equivalent, original code, as a series of imperative calls:

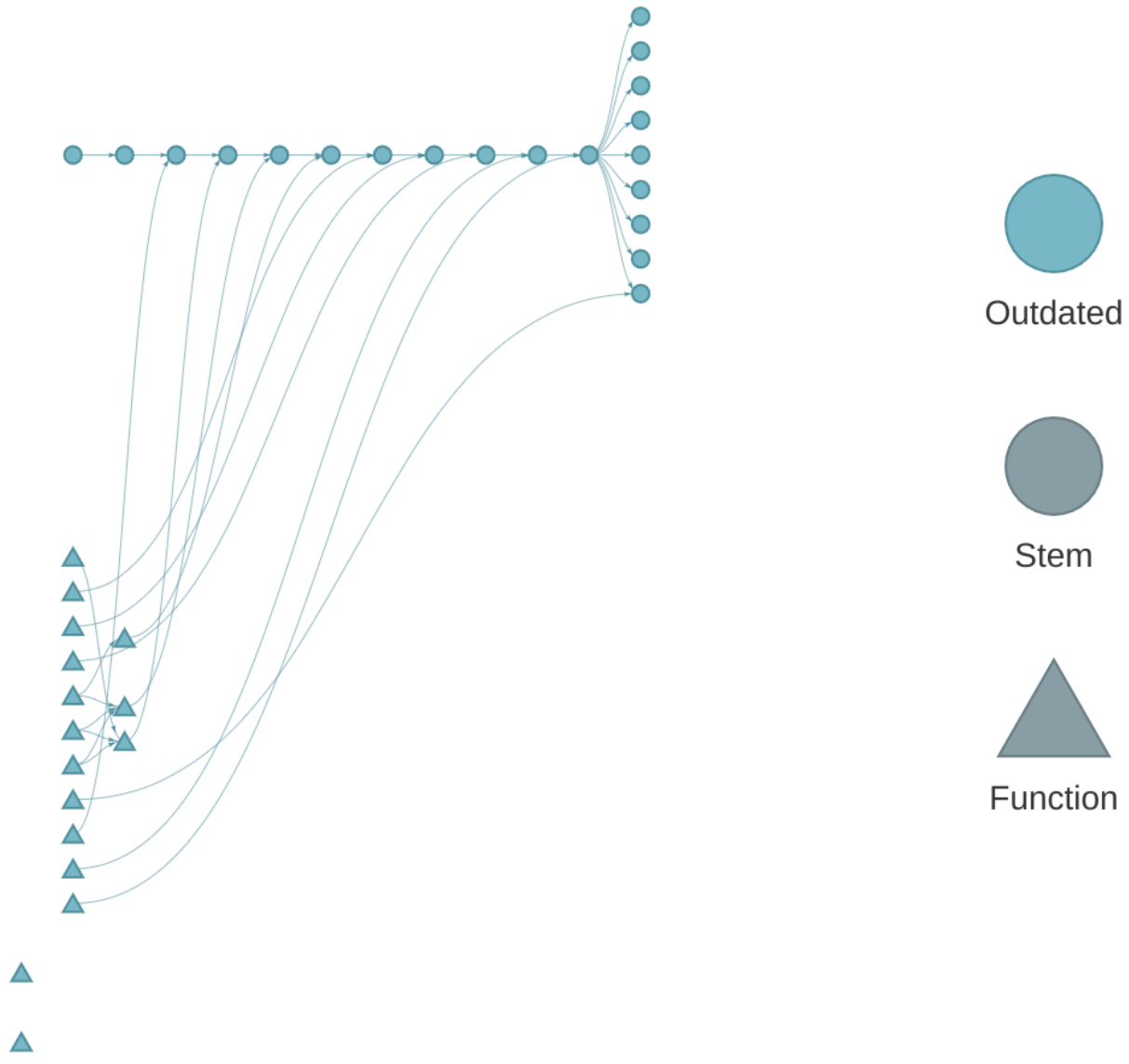
```
colnames(df) <- tolower(colnames(df))

# Exclude South Asians
df <- subset(df, (is.na(df$q24_2) | q24_2 == "No") &
                (is.na(df$q24_3) | q24_3 == "No"))

#############
## RECODES ##
#############

## RACIAL GROUP ##
df$race <- df$q_11rand
levels(df$race) <- c("WHITE-BLACK", "WHITE-ASIAN", "WHITE", "BLACK", "ASIAN")
df$race <- relevel(df$race, ref = "WHITE")
table(df$race)


## EDUCATION ##
df$educ3 <- recode(as.numeric(df$educ), "2=1; 3=2; 4=2; 5=3; 6=3")
df$educ3 <- factor(df$educ3, labels = c("Less than or equal HS", "Some College", "College Grad

## AGE ##
```

```
df$age <- 2015 - as.numeric(as.character(df$birthyr))


## INCOME ##
df$income <- ifelse(df$faminc == "Prefer not to say", NA, as.numeric(df$faminc))
df$income <- ifelse(df$income > 11, 12, df$income)
summary(df$income)


## REGION ##
southern_states <- c("Alabama", "Arkansas", "Florida", "Georgia", "Kentucky",
                     "Louisiana", "Mississippi", "North Carolina", "South Carolina",
                     "Tennessee", "Texas", "Virginia", "West Virginia")
df$south <- ifelse(df$inputstate %in% southern_states, 1, 0)
```

The original approach, which uses an imperative style of programming, keeps changing the `df` object, which is the original SPSS data loaded into memory. The issue with this approach is that it becomes very difficult to read and follow as the number of operations grows. It is also impossible to easily test imperative code. Functions on the other hand, are very easy to test, and very easy to chain together. Users interested in implementation details can look into the source code of the function to understand how it works. If not, reading `basic_clean_1(dataset)` is enough to convey that some basic cleaning operations were applied to `dataset`. There is no need to burden the user with too much code all at once.

The fourth target uses the output of the third target as an input and is defined as the output of the `closeness_2()` to function. Here again, readers that are interested into what `closeness_2()` does can look at its source code. If not, they can move on.

While defining the series of operations as a list of targets improves readability, it is sometimes much easier to look at a visual representation of the pipeline. This is possible using a command from the `{targets}` package called `tar_visnetwork()` Here is the visual representation of the pipeline, before actually running it:



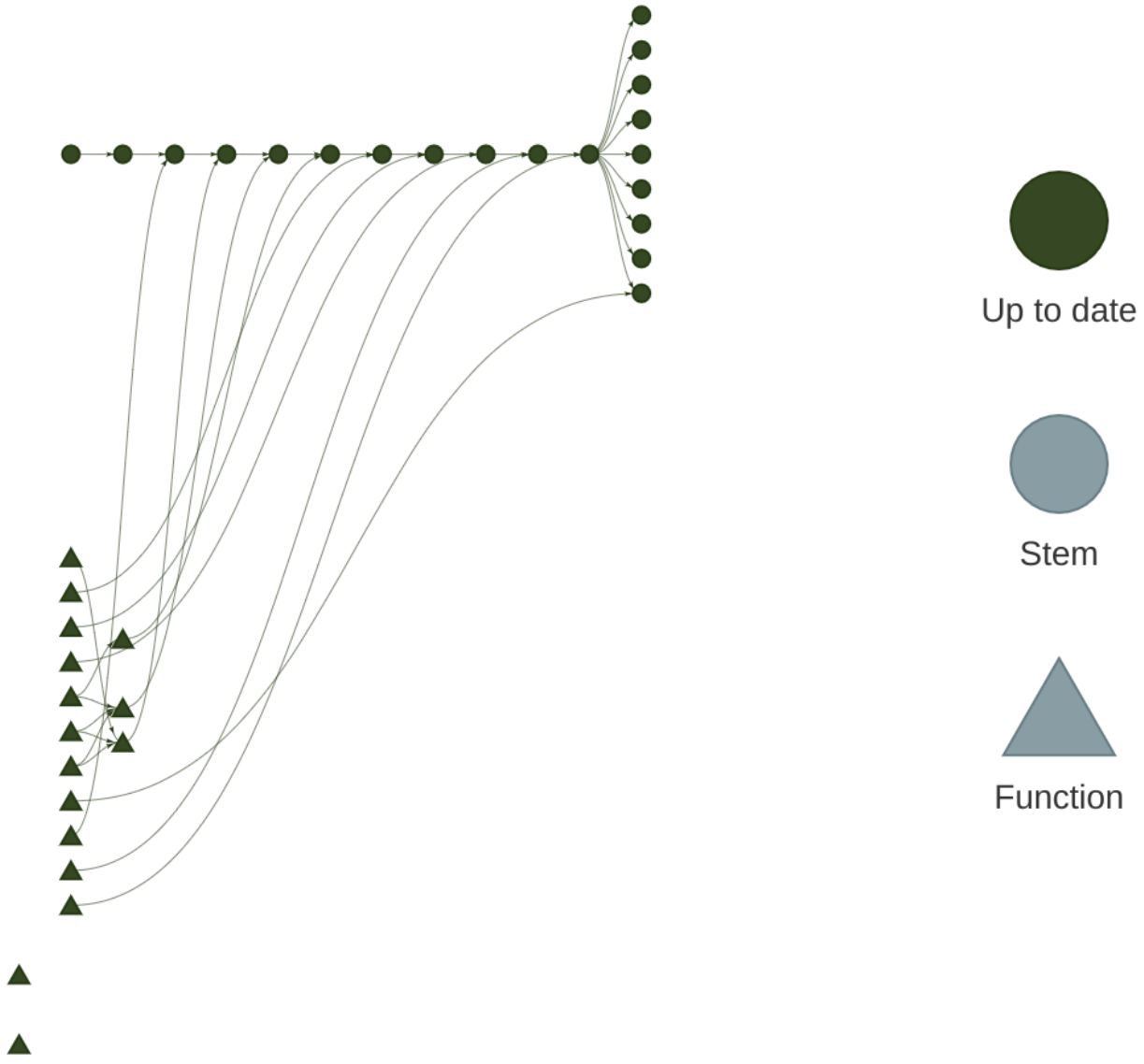
At a glance, it is possible to determine the following:

- a single dataset is being manipulated by several functions;
- each new instance of the dataset is the output of one, or several functions;
- there are two functions that were defined that are not being used at all (bottom left).

This is actually an interactive visualisation that opens in the users web-browser, so it is possible to zoom in and read the labels of the different objects.

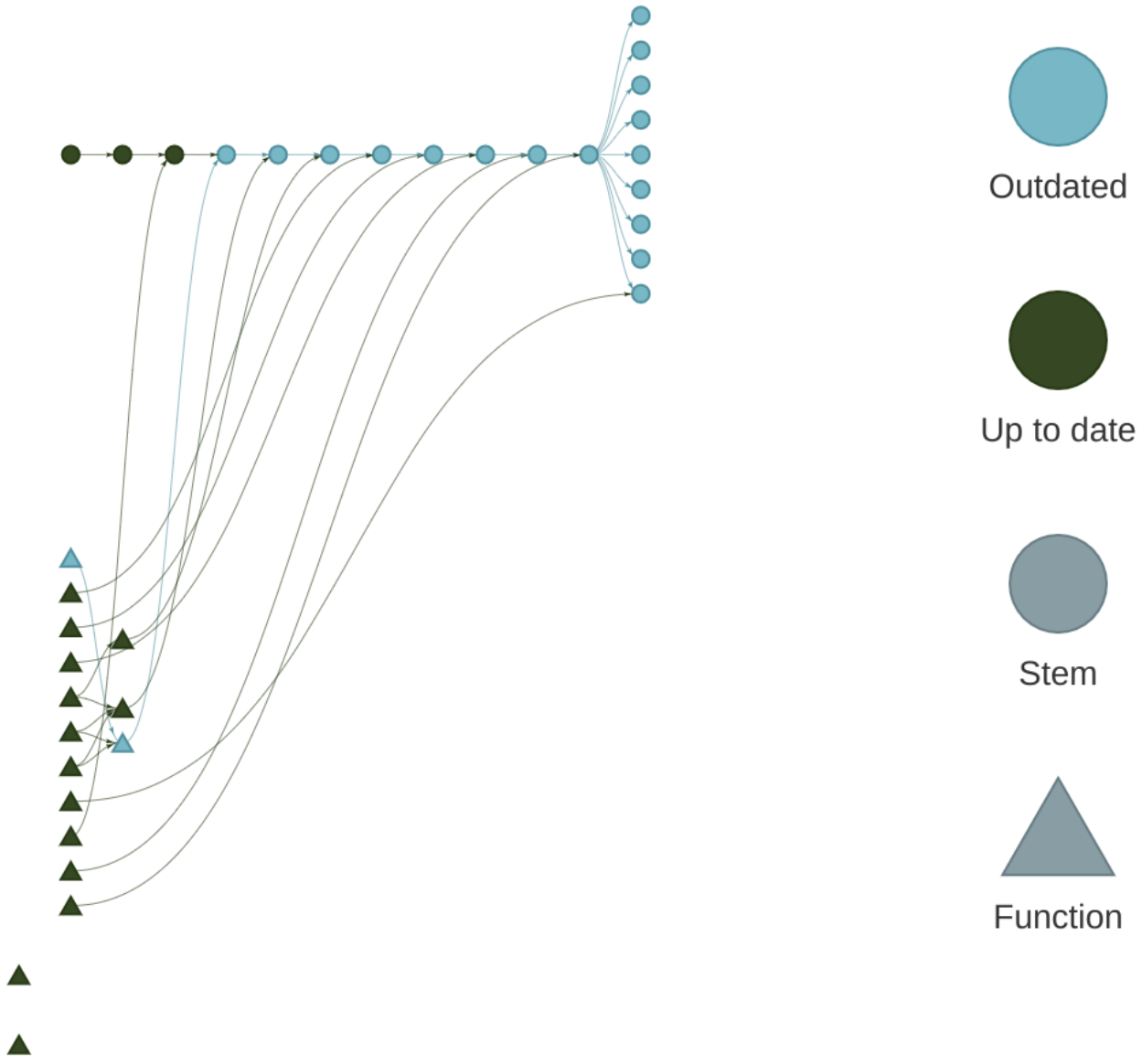
To run the pipeline, and compute all the targets, users must execute `targets::tar_make()`. This is how the

pipeline looks like once it's been executed:

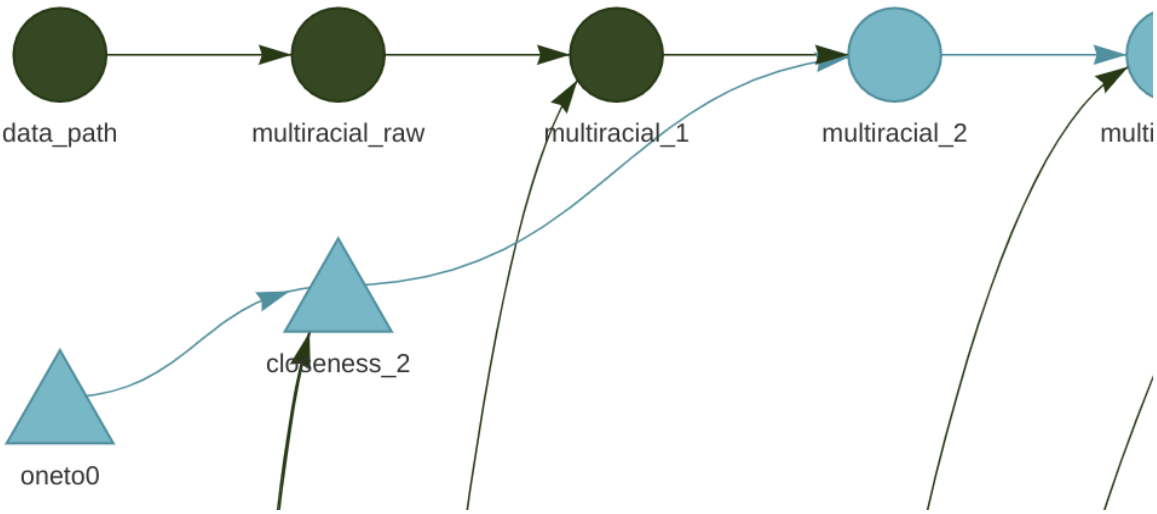


Each target is now updated, in other words, it has been computed. Any of the intermediary targets, as well as the very last target, `table_1`, can now be loaded into the R session in which the pipeline was executed using `targets::tar_load(table_1)`. Every target gets automatically saved into a folder that will appear after the first execution of the pipeline, called `_targets`. This folder contains all the targets, which means that even if I execute the pipeline in a completely new session the following day, the whole pipeline will get skipped, because all the targets have already been computed. This can lead to tremendous time savings. Also, if users do change some code, somewhere in the project, `{targets}` will automatically detect which line of code was changed, and what is the impact on the rest of the pipeline. For example, here we changed some code in one of

the functions and visualise the pipeline:



One function was changed, which impacted another function, which in turn impacted one of the intermediary datasets. Because this intermediary dataset was impacted, every other dataset down the pipeline will have to be recomputed up to the final target. Let's zoom in a bit:



The `oneto0()` function was changed (which is a piece of code that was also defined as a function in the original study), but because `closeness_2()` uses this function internally, it was thus impacted by that change. Finally, this means that the output of `closeness_2()` (`multiracial_2`) is also impacted, and thus needs to be recomputed, impacting every other target that uses `multiracial_2` as an input directly or indirectly.

In essence, `{targets}` forces its users to write small, well written functions and then chain them one after the next in order to generate an output. Visualising the execution of the code, and the impact that changing some pieces of code have on the whole pipeline helps readability immensely. Future users can start by looking at the `_targets.R` file, and then visualise the pipeline and get a high-level understanding of the whole computational process. If they want to read the technical implementation details of one or several functions, they can do so by simply going to the source code of the relevant function.

One final important note to improve reproducibility of a project: the output of scientific research is more often than not a PDF document. We highly recommend using literate programming (Knuth (1984)) coupled with `{targets}` to produce articles. In practice this means that the paper itself will get generated by `{targets}` as the final output of the whole pipeline. A fairly recent, but already very popular engine for literate programming that also supports LaTeXnotation is Quarto (Allaire et al. 2022).

0.4 Conclusion

Researchers have to deal with the following aspects of their projects to ensure their long-term reproducibility:

- They need to state the correct versions of R (or any other language) used for the analysis;
- They need to state the correct versions of every other used package for the analysis;
- They need to make sure that their code is readable and easy to explore
- They need to provide instructions on how to run the whole project.

By using Docker, it is possible to provide the correct versions of R and packages (using {renv}, {groundhog} or a snapshot of the CRAN repository), and by writing a pipeline using {targets} instead of using scripts, reseachers improve readability. But this is not only useful for turning projects reproducible: by using these tools, researchers also avoid common pitfalls while working on their projects. Research projects can last for years, and researchers typically work on several projects in parallel; by using the tools discussed in this article, researchers can thus have project-specific R and package versions, thus avoiding the common issue of updating their package library because they need the latest feature of a package for one project, but because packages got updated, another project now produces different results or simply does not run anymore. Using {targets} also helps researchers on a daily basis, as it improves the process of development. In other words, improving the reproducibility of a project is also useful for its development.

However, we realise that the entry cost to a tool like Docker is not low: so at the very least we advise researchers interested in reproducibility to be very conservative with updating R itself, and to use either {renv} or {groundhog} to have project-specific libraries. This is because if researchers at least provide the R version used and an renv.lock file (if they used {renv}, or a date, if they used {groundhog}) it is always possible to re-generate an environment with the right version of R and packages as we did for this paper. This solution is not the gold standard of reproducibility though, but easy enough to be implemented at a low cost. This also implies that researchers need to share their code and all other necessary information on a platform such as Github, and it implies that research has to be performed using free and open source tools as proprietary tools cannot easily be installed by other researchers wanting to reproduce a study.

Another issue with that we need to address is the over-reliance on Docker. While we have several alternatives for handling packages, Docker was the only solution that we suggested to essentially "freeze" versions of R and other lower-level system dependencies. We want to finish this conclusion by stating that there are other tools that achieve the same purpose, notably Nix (Dolstra 2006). Nix is not a containerisation tool like Docker, but a package manager that allows users to install exact versions of software, including scientific software, reproducibly. It is possible to essentially use Nix to replace Docker and `{renv}` (or `{groundhog}`), but presenting this tool is outside the scope of this article.

References

Allaire, J. J., Charles Teague, Carlos Scheidegger, Yihui Xie, and Christophe Dervieux. 2022. "Quarto." https://doi.org/10.5281/zenodo.5960048.

Aschwanden, Christie. 2015. "Psychology Is Starting To Deal With Its Replication Problem." *FiveThirtyEight*.

Bhandari Neupane, Jayanti, Ram P. Neupane, Yuheng Luo, Wesley Y. Yoshida, Rui Sun, and Philip G. Williams. 2019. "Characterization of Leptazolines a–d, Polar Oxazolines from the Cyanobacterium Leptolyngbya Sp., Reveals a Glitch with the 'Willoughby–Hoye' Scripts for Calculating NMR Chemical Shifts." *Organic Letters* 21 (20): 8449–53.

Broockman, David, Joshua Kalla, and Peter Aronow. 2015. "Irregularities in LaCour (2014)." *UC Berkeley. Http://Stanford. Edu/~ Dbroock/Broockman_kalla_aronow_lg_irregularities. Pdf*.

Camerer, Colin F, Anna Dreber, Eskil Forsell, Teck-Hua Ho, Jürgen Huber, Magnus Johannesson, Michael Kirchler, et al. 2016. "Evaluating Replicability of Laboratory Experiments in Economics." *Science* 351 (6280): 1433–36.

Chang, Andrew C., and Phillip Li. 2022. "Is Economics Research Replicable? Sixty Published Papers from Thirteen Journals Say 'Often Not'." *Critical Finance Review* 11 (1): 185–206. https://doi.org/10.1561/104.00000053.

Clemens, Michael A. 2017. "The Meaning of Failed Replications: A Review and Proposal." *Journal of Economic Surveys* 31 (1): 326–42.

Collaboration, Open Science. 2015. "Estimating the Reproducibility of Psychological Science." *Science* 349 (6251): aac4716.

Davenport, Lauren, Annie Franco, and Shanto Iyengar. 2022. "Multiracial identity and political preferences." *The Journal of Politics* 84 (1): 620–24.

Derksen, Maarten, and Jill Morawski. 2022. "Kinds of Replication: Examining the Meanings of 'Conceptual Replication' and 'Direct Replication'." *Perspectives on Psychological Science* 17 (5): 1490–1505.

Dolstra, Eelco. 2006. *The Purely Functional Software Deployment Model*. Utrecht University.

Gelman, Andrew. 2018. "Essay: The Experiments Are Fascinating. But Nobody Can Repeat Them." *The New York Times*, November.

Hamermesh, Daniel S. 2007. "Replication in Economics." *Canadian Journal of Economics/Revue Canadienne d'économique* 40 (3): 715–33.

Herndon, Thomas, Michael Ash, and Robert Pollin. 2014. "Does High Public Debt Consistently Stifle Economic Growth? A Critique of Reinhart and Rogoff." *Cambridge Journal of Economics* 38 (2): 257–79.

Knuth, Donald Ervin. 1984. "Literate Programming." *The Computer Journal* 27 (2): 97–111.

Landau, William Michael. 2021. "The Targets r Package: A Dynamic Make-Like Function-Oriented Pipeline Toolkit for Reproducibility and High-Performance Computing." *Journal of Open Source Software* 6 (57): 2959. https://doi.org/10.21105/joss.02959.

Lupia, Arthur, and Colin Elman. 2014. "Openness in political science: Data access and research transparency: Introduction." *PS: Political Science & Politics* 47 (1): 19–42.

Nosek, Brian A, and Timothy M Errington. 2020. "What Is Replication?" *PLoS Biology* 18 (3): e3000691.

Philip, Glandon. 2010. "Report on the American Economic Review Data Availability Compliance Project." *Unpublished Manuscript*.

Simonsohn, Uri. 2021. "[95] Groundhog: Addressing The Threat That R Poses To Reproducible Research." *Data Colada*. http://datacolada.org/95.

———. 2023. *Groundhog: Make Your r Scripts Reproducible by Replacing Library(pkg) with Groundhog.library(pkg,date)*.

Stockemer, Daniel, Sebastian Koehler, and Tobias Lentz. 2018. "Data access, transparency, and replication: New insights from the political behavior literature." *PS: Political Science & Politics* 51 (4): 799–803.